Vidyavardhini's College of Engineering & Technology
(Autonomous Institute Affiliated to University of Mumbai, Approved by AICTE & DTE)
NBA & NAAC Accredited
Department of Computer Science Engineering (Data Science)

EXPERIMENT ASSESSMENT

ACADEMIC YEAR 2026-27

Course: Agile Software Development and DevOps Lab

Course Code: 2015119

Year: TE SEM: V

Experiment No. 09
AIM:- To study and implement machine learning model development and experiment tracking using MLflow by recording model parameters, evaluation metrics, and trained models.
Name: Jay Chandrasen Talekar
Roll Number: 50
Date of Performance: 5/10/26
Date of Submission: 5/10/26

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission.	10	
Total	**20**	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	5	3	2
Understanding	5	3	2
Journal work and timely submission.	10	8	4

Checked by

Name of Faculty : Asst. Prof. Shilpa Bane

Signature :

Date :

Experiment No 09

MLflow Experiment Tracking Using Student Performance Dataset

Aim

To study and implement machine learning model development and experiment tracking using MLflow by recording model parameters, evaluation metrics, and trained models.

1. Problem Statement

A college wants to predict whether a student will **Pass or Fail** based on:

- Study Hours
- Attendance
- Internal Assessment Marks

We will develop a classification model and use **MLflow** to track the experiment.

2. Sample Dataset

Create a CSV file named:

student_performance.csv

Student_ID	Study_Hours	Attendance	Internal_Marks	Result
1	2	60	38	Fail
2	3	65	42	Fail
3	4	70	48	Pass
4	5	72	50	Pass
5	6	75	55	Pass
6	7	80	60	Pass
7	8	85	65	Pass
8	9	90	70	Pass
9	1	55	35	Fail
10	2	62	40	Fail

Student_ID	Study_Hours	Attendance	Internal_Marks	Result
11	3	68	45	Fail
12	5	74	52	Pass
13	6	78	57	Pass
14	7	82	62	Pass
15	8	88	68	Pass
16	4	69	47	Fail
17	3	64	41	Fail
18	6	76	56	Pass
19	7	84	63	Pass
20	9	92	72	Pass

Save this file in the **same folder as your Python/Jupyter Notebook**.

---

3. Tools Required

- Python 3.x
- Jupyter Notebook / VS Code
- MLflow
- Pandas
- Scikit-learn

Install the libraries:

!pip install mlflow pandas scikit-learn

---

4. Import Libraries

import pandas as pd

import mlflow

import mlflow.sklearn

from sklearn.model_selection import train_test_split

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

5. Load the Dataset

```
data = pd.read_csv("student_performance.csv")
print("Dataset:")
print(data)
print("\nDataset Shape:", data.shape)
```

Expected Output

Dataset:

	Student_ID	Study_Hours	Attendance	Internal_Marks	Result
0	1	2	60	38	Fail
1	2	3	65	42	Fail
2	3	4	70	48	Pass
3	4	5	72	50	Pass
4	5	6	75	55	Pass
5	6	7	80	60	Pass
6	7	8	85	65	Pass
7	8	9	90	70	Pass
8	9	1	55	35	Fail
9	10	2	62	40	Fail
10	11	3	68	45	Fail
11	12	5	74	52	Pass
12	13	6	78	57	Pass
13	14	7	82	62	Pass
14	15	8	88	68	Pass
15	16	4	69	47	Fail
16	17	3	64	41	Fail

17	18	6	76	56	Pass
18	19	7	84	63	Pass
19	20	9	92	72	Pass

Dataset Shape: (20, 5)

---

6. Separate Input and Output

We do not need Student_ID for prediction.

```
X = data[
    ["Study_Hours", "Attendance", "Internal_Marks"]
]


y = data["Result"]


print("Input Features:")
print(X.head())


print("\nTarget:")
print(y.head())
```

Expected Output

Input Features:

	Study_Hours	Attendance	Internal_Marks
0	2	60	38
1	3	65	42
2	4	70	48
3	5	72	50
4	6	75	55

Target:

0 Fail

1 Fail

2 Pass

3 Pass

4 Pass

Name: Result, dtype: object

7. Split the Dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))
```

Expected Output

Training Samples: 16

Testing Samples: 4

8. Create MLflow Experiment

```
mlflow.set_experiment("Student_Performance_Experiment")

print("MLflow Experiment Created Successfully")
```

Expected Output

MLflow Experiment Created Successfully

9. Train the Model and Track Using MLflow

We will use a **Decision Tree Classifier**.

```
max_depth = 3

with mlflow.start_run():

    # Create model
    model = DecisionTreeClassifier(
        max_depth=max_depth,
        random_state=42
    )

    # Train model
    model.fit(X_train, y_train)

    # Predict test data
    y_pred = model.predict(X_test)

    # Calculate accuracy
    accuracy = accuracy_score(y_test, y_pred)

    # Log parameters
    mlflow.log_param("model", "Decision Tree")
    mlflow.log_param("max_depth", max_depth)
    mlflow.log_param("random_state", 42)

    # Log metric
```

```
    mlflow.log_metric("accuracy", accuracy)

    # Log model
    mlflow.sklearn.log_model(
        model,
        "student_performance_model"
    )

    print("Model Training Completed")
    print("Accuracy:", accuracy)
```

Expected Output

With the given dataset, the output should be approximately:

Model Training Completed

Accuracy: 1.0

10. Display Actual Predictions

To make the experiment more meaningful for students, display the actual and predicted results.

```
result = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": y_pred
})

print(result)
```

Expected Output

The exact order depends on the train-test split, but students should get:

```
  Actual Predicted
0   Pass      Pass
1   Fail      Fail
```

2 Pass Pass

3 Pass Pass

Accuracy

Correct Predictions = 4

Total Predictions = 4

Accuracy = 4 / 4

= 1.00

= 100%

11. Start MLflow UI

Open **Command Prompt / Terminal** in the project folder.

Run:

mlflow ui

Expected message:

[INFO] Starting MLflow server...

[INFO] Listening at: http://127.0.0.1:5000

Open:

http://127.0.0.1:5000

12. Expected MLflow Dashboard

Students should see:

Experiment

Student_Performance_Experiment

Run Details

Parameter	**Value**
model	Decision Tree
max_depth	3

Parameter	Value
random_state	42

Metrics

Metric	Value
Accuracy	1.00

Artifacts

student_performance_model

The overall MLflow record will look conceptually like:

Student_Performance_Experiment

Run: Decision Tree

PARAMETERS

model Decision Tree

max_depth 3

random_state 42

METRICS

accuracy 1.000

ARTIFACTS

student_performance_model

13. Perform Second Experiment

Now students can change the model parameter:

max_depth = 2

Run the experiment again.

MLflow will create a **second run**.

The dashboard may show something like:

Run	Model	max_depth	Accuracy
Run 1	Decision Tree	3	1.00
Run 2	Decision Tree	2	0.75

The exact result may vary slightly based on the split and model.

What students learn

They can now answer:

Which model configuration performed better?

For example:

Run 1 → Accuracy = 100%

Run 2 → Accuracy = 75%

Therefore, Run 1 performed better.

This is the main purpose of **MLflow experiment tracking**.

14. Expected Final Output

At the end of the experiment, students should obtain:

Dataset Shape: (20, 5)

Training Samples: 16

Testing Samples: 4

Model Training Completed

Accuracy: 1.0

Actual vs Predicted:

	Actual	Predicted
0	Pass	Pass
1	Fail	Fail
2	Pass	Pass
3	Pass	Pass

And in MLflow:

Experiment:

Student_Performance_Experiment

Parameters:

model = Decision Tree

max_depth = 3

random_state = 42

Metric:

accuracy = 1.00

Artifact:

student_performance_model

15. Result

The Decision Tree classification model was successfully developed using the student performance dataset. The model achieved an accuracy of approximately **100% on the test samples**. The model parameters, accuracy metric, and trained model were successfully tracked and stored using MLflow.

Code:

```
import os


# Fix MLflow file store restriction and silence Git warnings
os.environ["MLFLOW_ALLOW_FILE_STORE"] = "true"
os.environ["GIT_PYTHON_REFRESH"] = "quiet"


import pandas as pd
import mlflow
import mlflow.sklearn
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score


# Step 1: Set MLflow experiment name
mlflow.set_experiment("Student_Performance_Experiment")


# Step 2: Create dataset CSV if it does not exist
csv_file = "student_performance.csv"
if not os.path.exists(csv_file):
    dataset_content =
"""Student_ID,Study_Hours,Attendance,Internal_Marks,Result
1,2,60,38,Fail
2,3,65,42,Fail
3,4,70,48,Pass
4,5,72,50,Pass
5,6,75,55,Pass
6,7,80,60,Pass
7,8,85,65,Pass
```

```
8,9,90,70,Pass
9,1,55,35,Fail
10,2,62,40,Fail
11,3,68,45,Fail
12,5,74,52,Pass
13,6,78,57,Pass
14,7,82,62,Pass
15,8,88,68,Pass
16,4,69,47,Fail
17,3,64,41,Fail
18,6,76,56,Pass
19,7,84,63,Pass
20,9,92,72,Pass"""
    with open(csv_file, "w") as f:
        f.write(dataset_content.strip())

# Step 3: Load dataset
data = pd.read_csv(csv_file)
X = data[["Study_Hours", "Attendance", "Internal_Marks"]]
y = data["Result"]

# Step 4: Train-test split (80% Train, 20% Test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)


def run_experiment(max_depth, run_name):
    with mlflow.start_run(run_name=run_name):
```

```
        model = DecisionTreeClassifier(max_depth=max_depth,
random_state=42)
        model.fit(X_train, y_train)


        y_pred = model.predict(X_test)
        accuracy = accuracy_score(y_test, y_pred)


        mlflow.log_param("model", "Decision Tree")
        mlflow.log_param("max_depth", max_depth)
        mlflow.log_param("random_state", 42)


        mlflow.log_metric("accuracy", accuracy)


        # Updated model logging with trusted tree structure types
        mlflow.sklearn.log_model(
            sk_model=model,
            artifact_path="student_performance_model",
            skops_trusted_types=["sklearn.tree._tree.Tree"],
        )


        print("=" * 55)
        print(f" LOGGED RUN: {run_name}")
        print("=" * 55)
        print(f"max_depth : {max_depth}")
        print(f"Accuracy  : {accuracy * 100:.2f}%\n")


        comparison = pd.DataFrame({"Actual": y_test.values,
"Predicted": y_pred})
        print(comparison)
        print("-" * 55 + "\n")
```

```python
if __name__ == "__main__":
    run_experiment(max_depth=3, run_name="Run_1_Depth_3")
    run_experiment(max_depth=2, run_name="Run_2_Depth_2")
    print("Experiment tracking complete!")
```

OUTPUT:

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\admin\Desktop\TE#> python exp.py
2026/10/07 17:00:13 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
🏃 LOGGED RUN: Run_1_Depth_3
max_depth : 3
Accuracy  : 100.00%

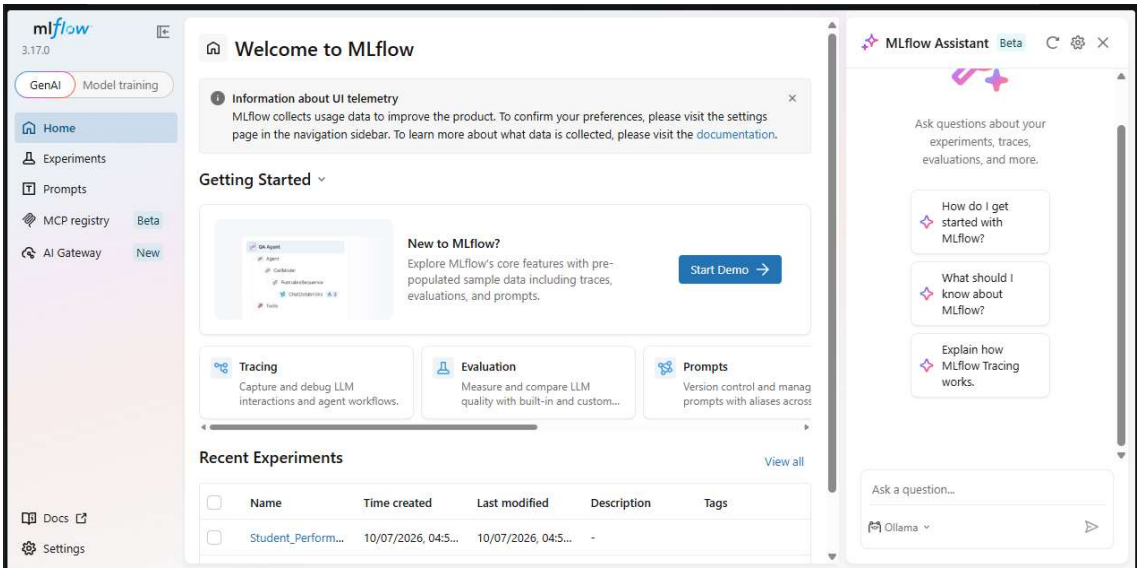
  Actual Predicted
0   Pass      Pass
1   Fail      Fail
2   Pass      Pass
3   Pass      Pass
----------

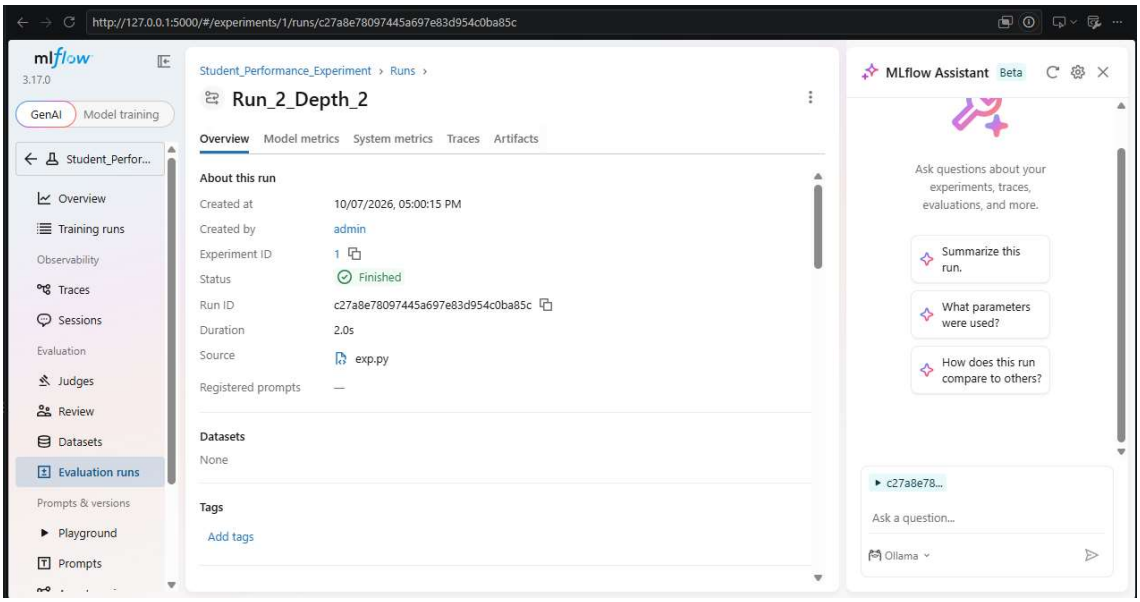
2026/10/07 17:00:15 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
🏃 LOGGED RUN: Run_2_Depth_2
max_depth : 2
Accuracy  : 100.00%

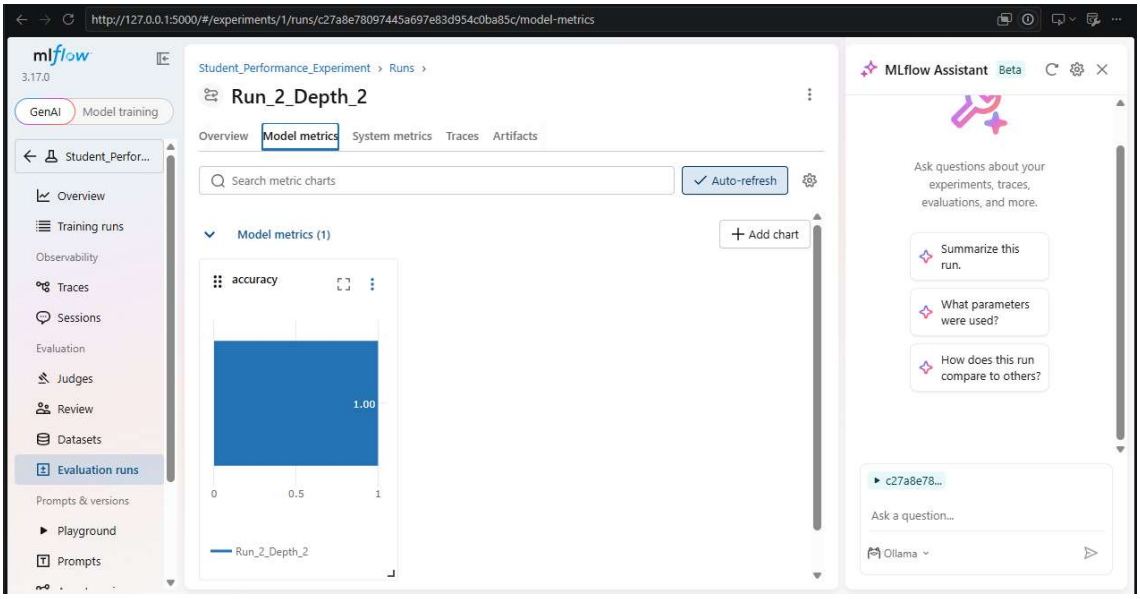
  Actual Predicted
0   Pass      Pass
1   Fail      Fail
2   Pass      Pass
3   Pass      Pass
----------

Experiment tracking complete!
```

```
PS C:\Users\admin\Desktop\TE#> mlflow ui
Backend store URI not provided. Using sqlite:///mlflow.db
Registry store URI not provided. Using backend store URI.
[MLflow] Security middleware enabled with default settings (localhost-only). To allow connections from other hosts, use --host 0.0.0.0 and configure --allowed-
hosts and --cors-allowed-origins.
2026/10/07 17:00:24 WARNING mlflow.server: MLflow job execution requirements not met (MLflow job backend does not support Windows system.). Server will start w
ithout job execution support. Errors will be surfaced at job invocation time.
2026/10/07 17:00:24 INFO:     Uvicorn running on http://127.0.0.1:5000 (Press CTRL+C to quit)
2026/10/07 17:00:24 INFO:     Started parent process [15864]
C:\Users\admin\AppData\Local\Programs\Python\Python314\Lib\site-packages\mlflow\server\fastapi_app.py:23: StarletteDeprecationWarning: starlette.middleware.wsg
i is deprecated and will be removed in a future release. Please refer to https://github.com/abersheeran/a2wsgi as a replacement.
  from starlette.middleware.wsgi import WSGIResponder, build_environ
C:\Users\admin\AppData\Local\Programs\Python\Python314\Lib\site-packages\mlflow\server\fastapi_app.py:23: StarletteDeprecationWarning: starlette.middleware.wsg
i is deprecated and will be removed in a future release. Please refer to https://github.com/abersheeran/a2wsgi as a replacement.
  from starlette.middleware.wsgi import WSGIResponder, build_environ
C:\Users\admin\AppData\Local\Programs\Python\Python314\Lib\site-packages\mlflow\server\fastapi_app.py:23: StarletteDeprecationWarning: starlette.middleware.wsg
i is deprecated and will be removed in a future release. Please refer to https://github.com/abersheeran/a2wsgi as a replacement.
  from starlette.middleware.wsgi import WSGIResponder, build_environ
C:\Users\admin\AppData\Local\Programs\Python\Python314\Lib\site-packages\mlflow\server\fastapi_app.py:23: StarletteDeprecationWarning: starlette.middleware.wsg
i is deprecated and will be removed in a future release. Please refer to https://github.com/abersheeran/a2wsgi as a replacement.
  from starlette.middleware.wsgi import WSGIResponder, build_environ
2026/10/07 17:00:26 INFO:     Started server process [22096]
2026/10/07 17:00:26 INFO:     Waiting for application startup.
2026/10/07 17:00:26 INFO:     Started server process [20784]
2026/10/07 17:00:26 INFO:     Started server process [23408]
2026/10/07 17:00:26 INFO:     Waiting for application startup.
2026/10/07 17:00:26 INFO:     Waiting for application startup.
2026/10/07 17:00:26 INFO:     Application startup complete.
2026/10/07 17:00:26 INFO:     Application startup complete.
2026/10/07 17:00:26 INFO:     Application startup complete.
2026/10/07 17:00:26 INFO:     Started server process [12084]
2026/10/07 17:00:26 INFO:     Waiting for application startup.
2026/10/07 17:00:26 INFO:     Application startup complete.
2026/10/07 17:00:32 INFO:     127.0.0.1:54892 - "GET / HTTP/1.1" 304 Not Modified
2026/10/07 17:00:34 INFO:     127.0.0.1:54892 - "GET /ajax-api/3.0/mlflow/server-info HTTP/1.1" 200 OK
2026/10/07 17:00:34 INFO:     127.0.0.1:54892 - "GET /ajax-api/2.0/mlflow/users/current HTTP/1.1" 404 Not Found
2026/10/07 17:00:34 INFO:     127.0.0.1:55571 - "GET /ajax-api/3.0/mlflow/assistant/config HTTP/1.1" 200 OK
2026/10/07 17:00:34 INFO:     127.0.0.1:63054 - "GET /ajax-api/3.0/mlflow/assistant/providers HTTP/1.1" 200 OK
2026/10/07 17:00:34 INFO:     127.0.0.1:55571 - "GET /ajax-api/2.0/mlflow/experiments/search?max_results=5&order_by=last_update_time+DESC HTTP/1.1" 200 OK
2026/10/07 17:01:10 INFO:     127.0.0.1:62725 - "GET /static-files/static/js/6507.352ad2e0.chunk.js HTTP/1.1" 200 OK
2026/10/07 17:01:10 INFO:     127.0.0.1:51208 - "GET /static-files/static/css/5439.24118451.chunk.css HTTP/1.1" 200 OK
2026/10/07 17:01:10 INFO:     127.0.0.1:51208 - "POST /ajax-api/2.0/mlflow/runs/search HTTP/1.1" 200 OK
2026/10/07 17:01:10 INFO:     127.0.0.1:51208 - "GET /static-files/static/js/5439.b5816bbb.chunk.js HTTP/1.1" 200 OK
2026/10/07 17:01:10 INFO:     127.0.0.1:63524 - "GET /static-files/static/js/9861.2d6b3bae.chunk.js HTTP/1.1" 200 OK
2026/10/07 17:01:10 INFO:     127.0.0.1:62725 - "POST /graphql HTTP/1.1" 200 OK
2026/10/07 17:01:10 INFO:     127.0.0.1:51419 - "GET /static-files/static/js/353.df7e2039.chunk.js HTTP/1.1" 200 OK
2026/10/07 17:01:10 INFO:     127.0.0.1:51210 - "GET /static-files/static/js/2809.4115baa4.chunk.js HTTP/1.1" 200 OK
2026/10/07 17:01:10 INFO:     127.0.0.1:63138 - "GET /static-files/static/js/3448.da9a79ba.chunk.js HTTP/1.1" 200 OK
2026/10/07 17:01:10 INFO:     127.0.0.1:63524 - "GET /static-files/static/js/9538.46130387.chunk.js HTTP/1.1" 200 OK
2026/10/07 17:01:10 INFO:     127.0.0.1:63138 - "GET /static-files/static/js/1416.113691cc.chunk.js HTTP/1.1" 200 OK
2026/10/07 17:01:10 INFO:     127.0.0.1:51419 - "GET /static-files/static/js/6395.ad9cfe58.chunk.js HTTP/1.1" 200 OK
2026/10/07 17:01:10 INFO:     127.0.0.1:51210 - "GET /static-files/static/js/9848.664e1dcc.chunk.js HTTP/1.1" 200 OK
```




http://127.0.0.1:5000/#/experiments/1/runs/c27a8e78097445a697e83d954c0ba85c
mlflow
3.17.0
GenAI
Model training
Student_Perfor...
Overview
Training runs
Observability
Traces
Sessions
Evaluation
Judges
Review
Datasets
Evaluation runs
Prompts & versions
Playground
Prompts
Student_Performance_Experiment › Runs ›
Run_2_Depth_2
Overview
Model metrics
System metrics
Traces
Artifacts
About this run
Created at
10/07/2026, 05:00:15 PM
Created by
admin
Experiment ID
1
Status
Finished
Run ID
c27a8e78097445a697e83d954c0ba85c
Duration
2.0s
Source
exp.py
Registered prompts
—
Datasets
None
Tags
Add tags
MLflow Assistant
Beta
Ask questions about your experiments, traces, evaluations, and more.
Summarize this run.
What parameters were used?
How does this run compare to others?
c27a8e78...
Ask a question...
Ollama


http://127.0.0.1:5000/#/experiments/1/runs/c27a8e78097445a697e83d954c0ba85c/model-metrics
mlflow
3.17.0
GenAI
Model training
Student_Perfor...
Overview
Training runs
Observability
Traces
Sessions
Evaluation
Judges
Review
Datasets
Evaluation runs
Prompts & versions
Playground
Prompts
Student_Performance_Experiment › Runs ›
Run_2_Depth_2
Overview
Model metrics
System metrics
Traces
Artifacts
Search metric charts
Auto-refresh
Model metrics (1)
Add chart
accuracy
1.00
0
0.5
1
Run_2_Depth_2
MLflow Assistant
Beta
Ask questions about your experiments, traces, evaluations, and more.
Summarize this run.
What parameters were used?
How does this run compare to others?
c27a8e78...
Ask a question...
Ollama